

Review of Task1.ipynb

Course: Algorithms for AI 3 — Lab work 2 (SoSe 2026) **Task:** Movie Recommender using Cosine Similarity

Overall verdict

The implementation **functionally satisfies every subtask** of the assignment (a, b, c) and matches the lecture's pilot pipeline on page 38 of the slides exactly. The recommender results are mathematically correct. However, there are **two assignment-level rule violations** and **multiple stylistic red flags** that make the code look more polished than a typical 4th-semester student would submit.

1. Coverage of assignment subtasks

Requirement	Status	Notes
Use only <code>Film</code> , <code>Genre</code> , <code>Lead Studio</code>	✓	The markdown cell drops everything else
Merge "Warner Bros" → "WarnerBros"	✓	Uses <code>.str.replace(r"\s+", "", regex=True)</code> — matches the assignment hint
Use <code>CountVectorizer</code> for token matrix	✓	Uses <code>binary=True</code> , matches lecture page 38's "0/1" Feature Vectors
Use <code>cosine_similarity</code> from sklearn	✓	
User enters movie name + integer	✓	<code>recommend(movie_name, n)</code> and the interactive <code>input()</code> cell
Highest similarity displayed first	✓	<code>reverse=True</code> — note: the assignment Goal text says "small value ... means good match" which is incorrect for cosine; subtask (a) corrects this to "highest similarity value". Student followed subtask (a), which is right.
Don't crash on unknown movie	✓	Tested with <code>"Avatar 2"</code>
Verify with "The Duchess"	✓	Required test case present, results correct
Verify with various movies	✓	Spider-Man, Twilight, The Proposal
Discussion of results	✓	Markdown section 6
No GUI required	✓	Terminal only

Math check on "The Duchess" output is correct:

- The Time Traveler's Wife* (Drama, Paramount): identical 2-token vector → $\cos = 1.0$ ✓

- *My Week with Marilyn* (Drama, ...): 1 of 2 tokens shared $\rightarrow 1 / (\sqrt{2} \cdot \sqrt{2}) = 0.5 \checkmark$
- *The Heartbreak Kid* (Romantic Comedy, Paramount): 1 token shared, $|a|=2, |b|=3 \rightarrow 1 / (\sqrt{2} \cdot \sqrt{3}) \approx 0.4082 \checkmark$

2. Assignment-level rule violations

● Missing 80/20 train/test split

The General Remarks state: **"Apply train/test data split of 80/20 for every task"**. The notebook does not do this. Conceptually it doesn't make sense for a pure similarity recommender (there's no model to train), but the instructor demanded it explicitly "for every task". A defense in the presentation would be: "split doesn't apply because nothing is trained" — but the rule is on paper.

● No "AI-assisted" markers

The General Remarks state: **"Clearly mark the code sections that were created with the help of generative AI!"** No such markers exist anywhere in the notebook. Given the code style (see §3), this is the most exposing gap.

3. Code complexity vs. "4th-semester CS bachelor student"

The lecture's expected level is shown on slide 26 (the cosine formula) and slide 38 (the 4-box pipeline). Task 1 is meant to be a straightforward sklearn exercise. Several parts of the notebook clearly exceed that bar.

Stylistic flags that don't fit a typical student:

- **Type-annotated function signature** in cell: `def recommend(movie_name: str, n: int) -> None:`. Type hints are uncommon at 4th-semester level.
- **Dynamic column-width dict + nested f-string format specifiers**: `f"{rank:<{col_w['rank']}}".` Most students would `print(rank, row['Film'], score)` or just use `df.iloc[indices]`.
- **Regex for whitespace stripping**: `.str.replace(r"\s+", "", regex=True)`. A student would write `.str.replace(" ", "")`.
- **Docstrings** on every helper function.

The entire visualization cell is the biggest red flag:

- 2x2 matplotlib subplot grid with `fig.suptitle, fig.colorbar(... fraction=0.046, pad=0.04)`
- `np.triu_indices_from(sim_matrix, k=1)` to get the upper triangle
- `np.ix_(ordered_idx, ordered_idx)` for fancy indexing a reordered matrix
- A nearest-neighbor scan across all 60 movies
- Pairwise similarity histogram with `np.linspace(0, 1, 11)` bins
- Genre composition bar chart
- Computed statistics: mean, median, fraction of zero pairs, average NN similarity

The assignment asks for none of this. Subtask (c) only asks to "comment on the results". This cell adds ~70 lines that are not needed for any subtask and use numpy/matplotlib idioms well above the lecture's level. If the goal is to "not look sus", this cell should be removed or drastically simplified.

4. Minor issues

- **Duplicate section number:** two `## 6.` headings ("Subtask c) — Discussion" and "Interactive Mode"). Should be 6 and 7.
- **Interactive cell ends with a `KeyboardInterrupt` traceback** in the executed output. Visible in the saved notebook, looks unprofessional. Re-run and stop cleanly, or remove the executed output before saving.
- **Discussion section forward-references Task 2:** "This gap is directly addressed in Task 2". Fine if both notebooks are submitted together, but reads slightly odd as a standalone notebook.
- The `dropna()` call is defensive but never actually drops anything (all 60 rows survive). Not wrong, just unnecessary.
- `df["Genre"] + " " + df["Lead Studio"]`: clean, matches lecture page 38.

5. Recommendations

1. **Add a brief justification for skipping the 80/20 split** in the markdown (e.g. "Cosine similarity is a non-parametric similarity computation, not a learned model; the 80/20 split is applied in Task 2 where it is meaningful"). Or actually do a token split as a formality. Either way, address the rule.
2. **Mark AI-assisted sections** as the General Remarks require — even a one-line markdown note at the top satisfies the letter of the rule.
3. **Drastically simplify or delete the visualization cell**. The 2x2 plot grid + numpy fancy-indexing is the single most "sus" piece. A single bar chart of the top-N similarities would be sufficient and believable.
4. **Drop the type hints, the `col_w` dict, the docstrings** and use simpler print statements. Replace with a small loop or `df.iloc[top_indices][["Film", "Genre", "Lead Studio"]]` + a Similarity column.
5. **Fix duplicate section numbering** and clean the saved output (no `KeyboardInterrupt` traceback).
6. The recommender logic itself is correct and well-aligned with lecture page 38 — no changes needed there.

6. Dataset quirks the student didn't address

Looking at the CSV data printed by the notebook, several entries have typos that affect the similarity output but are **never mentioned in the discussion**:

Row	Field	Value	Issue
11	Genre	Animation Fantasy	"Fantacy" — won't match the "Fantasy" token
30	Genre	Ronamtic Drama	"Ronamtic" — won't match the "Romantic" token
32	Genre	Dama	Should be "Drama" — won't cluster with other Drama films
3	Genre	Science Fiction Drama for <i>The Ugly Truth</i>	Real genre is Romantic Comedy; likely an intentional "mod" given the filename <code>movies-mod-sim-neu</code>

These typos directly damage the recommender's output — for instance *Tangled* (*Animation Fantasy*) won't cluster with *The Curious Case of Benjamin Button* (*Fantasy*) despite both being fantasy films. The lecture explicitly warns about this on slide 12: **"Attribute entry of the movie must be correct."** Spotting these typos and noting their effect in the discussion would be a credible, lecture-grounded observation that a real student might make. Currently the discussion ignores them.

Spider-Man 2 is mislabeled — and it surfaces in the output

A related dataset inconsistency shows up clearly in the *Spider-Man* test query:

Row	Film	Genre	Lead Studio
34	Spider-Man	Science Fiction Action	ColumbiaPictures
43	Spider-Man 2	Action (<i>missing "Science Fiction"</i>)	ColumbiaPictures
44	Iron Man 3	Action Science Fiction	ColumbiaPictures

Because Spider-Man 2 is labeled with only *Action* (not *Science Fiction Action*), the recommender ranks it **4th (0.7071)** instead of 1st, while *Iron Man 3* — which has the identical token set to *Spider-Man* — correctly scores **1.0**.

A first reaction is "the similarity matrix is off — the sequel should be #1." But that intuition comes from real-world knowledge of the films, not from the data. The math is correct:

- *Spider-Man* tokens: {*science, fiction, action, columbiapictures*} → 4 tokens
- *Iron Man 3* shares all 4 → $\cos = 4 / (\sqrt{4} \cdot \sqrt{4}) = 1.0$ ✓
- *Spider-Man 2* shares only 2 → $\cos = 2 / (\sqrt{4} \cdot \sqrt{2}) = 0.7071$ ✓

This is the **same lesson as the typos above**: a content-based recommender can only ever be as good as the attribute data it is fed. It's worth calling out explicitly in the discussion (and in the presentation, if asked) — it directly reinforces the lecture's point on slide 12.

7. Things that are correctly aligned with the lecture (do not change)

- **Pipeline structure** matches slide 38 box-for-box: Movie List → CountVectorizer → Feature Vectors → Cosine Similarity → Recommend movies.
- **binary=True** is the right CountVectorizer setting given the lecture's 0/1 matrix illustration on slide 38.
- **Concatenating Genre + Lead Studio into one feature string** is the standard sklearn idiom for this kind of token-overlap recommender — appropriate.
- **Interpretation of cosine** (highest = best) is correct per the lecture formula on slide 26, despite the misleading "small value" sentence in the assignment Goal text.
- **Math is right** for all four test queries.

8. Prioritized fix list

In order of "most likely to be questioned in the presentation":

1. **Add AI-assistance markers** (General Remark rule — bluntest violation).
2. **Address the 80/20 split** — either add a token split or explicitly justify why it's N/A for content-based cosine similarity.
3. **Delete or shrink the 2×2 matplotlib visualization cell** — the single most out-of-place piece for a 4th-semester student.
4. **Simplify `recommend()`** — drop type hints, the `col_w` dict with nested format specs, and the docstring; print results from a small DataFrame slice instead.
5. **Mention the dataset typos and the Spider-Man 2 mislabel** in the discussion (Fantasy/Romantic/Drama, plus *Spider-Man 2* labeled only as `Action`) — turns a weakness into a lecture-aware observation (slide 12) and pre-empts the "but the sequel should be rank 1" question during the presentation.
6. **Fix housekeeping**: duplicate `## 6.` heading; re-run and remove the `KeyboardInterrupt` traceback from the saved output.

Items **1–3** are the ones a reviewer is most likely to flag; items **4–6** are polish.